FPT UNIVERSITY PRM393 MOBILE PROGRAMMING

LAB 3: CLOUD-ENABLED MOBILE APP

EXEMatching: Cloud-Enabled Capstone Team Formation with Firebase, Google ARTEMIS & AI Code Review

PROJECT METADATA

Team: DEVORGANIZE222
App: EXEMatching (Android)
Firebase ID: prm393-lab-3-exematching
Package: com.devorganize222.exematching
Target SDK: Android 16 (API 36)
Flutter SDK: 3.47.4 / Dart 3.13.3

EXECUTIVE SUMMARY

This report documents the transformation of the Lab 2 Figma prototype into a cloud-enabled Android application powered by Google Firebase. The application adheres to strict Model-View-ViewModel (MVVM) architecture with Provider, enforces zero Firebase SDK calls from UI widgets, and verifies all 3 user flows end-to-end.

Key Deliverables: 10 production screens, 6 integrated Firebase cloud services, 10 privacy-compliant Analytics events, 13 automated E2E tests driven by Google ARTEMIS (24/39 runs passed), and an AI code review with 5 findings across 3 categories.

1. PROJECT OVERVIEW & LAB 2 RECAP

 Persona: Minh Le, a senior Software Engineering student needing to form an official 4-5 member capstone team before the semester registration deadline.
 Success Criterion: Students can find an open team matching their skills, elect a leader with hidden voting tallies, and the leader locks the roster, recovering from under-capacity via Waiting Pool automated invites.
 Three Critical User Flows:
  - Flow 1 (Discovery & Join): Browse teams with skill filters, view team details, join open role with capacity validation.
  - Flow 2 (Leader Election): Members cast confidential votes; real-time tally reveals winner upon majority consensus.
  - Flow 3 (Roster Finalization): Leader validates team conditions (4-5 members, supervisor, leader). If under-capacity, leader triggers automated invitations from Waiting Pool, then locks roster to generate official code.

2. DESIGN TRACEABILITY & SYSTEMATIC DEVIATIONS

Figma PRM_LAB2 (Page 03 Final UI) -> Flutter Code Mapping

All 9 Figma screens plus an added Firebase Profile Screen have been implemented in Flutter with full state fidelity (Loading, Empty, Error, Data). Deviations are governed by the UX Writing Checklist (Bobbie Wood) and UI Consistency Checklist.

Screen ID	Figma Node	Flutter Route	ViewModel	Data Source	States Verified
SCR_01	Sign In (FPT SSO)	/sign-in	SignInViewModel	AuthService	Default, SigningIn, Cancelled, WrongDomain
SCR_02	Dashboard / Home	/home	DashboardViewModel	TeamRepo, RegService	Loading, InTeam, NoTeam, RegClosed, Error
SCR_03	Browse Groups	/browse	BrowseViewModel	TeamRepo, ConfigService	Loading, Results, Empty, OfflineError, Retry
SCR_04	Group Detail	/browse/team/:id	TeamDetailViewModel	TeamRepository	Loading, Default, Joining, FilledError, Member
SCR_05	My Group Hub	/team	MyTeamViewModel	TeamRepository	Loading, NoTeam, ElectLeader, ReadyLock, Locked
SCR_06	Leader Voting	/team/vote	VotingViewModel	TeamRepository	Loading, Unvoted(Hidden), Selected, Elected
SCR_07	Lock & Recovery	/team/lock	LockReviewViewModel	TeamRepository	Loading, Blocked(<4), Inviting, Recovered, Locked
SCR_08	Random Pool	/home/pool	PoolStatusViewModel	PoolRepository	Loading, Matching, QueueStatus, Error
SCR_09	Alerts & Invites	/alerts	NotificationsViewModel	NotificationRepo	Loading, List, Filtered, Empty, Decline+Undo
Profile	Firebase Demo Area	/profile	ProfileViewModel	Storage, Config, Crash	Default, PDF Exporting, Progress, URL, Crash

Key Systematic Deviations & Technical Rationale:

- DV-01 (Terminology): Unified "Group" and "Team" to "Team" everywhere for cognitive clarity.
- DV-04 (Direct Join): In Figma, join sent an unhandled request; app joins directly upon open-slot validation.
- DV-05 (Actionable Errors): Changed vague "Ineligible to Lock" to front-loaded "Can't lock yet: needs 1 more member".
- DV-08 (Profile / Demo Area): Repurposed top bar avatar to open Profile screen hosting all Firebase demo actions.
- DV-09/10 (WCAG Accessibility): Boosted warning text to #B4530A (5.7:1 contrast) and inputs to #737980 (>=3:1).

3. ARCHITECTURE & MVVM IMPLEMENTATION

Strict Layer Separation, State Modeling & Dependency Injection

The codebase enforces unidirectional downward dependencies: UI (View) -> ViewModel -> Service Interface -> Concrete Firebase Implementations. No widget contains business logic or direct SDK imports.

ARCHITECTURAL PRINCIPLES (Section 5 Compliance):

1. Zero SDK Leakage: lib/screens/ and lib/widgets/ never import firebase_core, cloud_firestore, firebase_auth, or http. All external operations are encapsulated in lib/firebase/.
2. Composition Root: lib/app/providers.dart creates concrete singletons and binds them to abstract interfaces via Provider. Swapping cloud backends requires no UI changes.
3. Explicit State Modeling: Every data screen is governed by sealed class ViewState<T> (Loading, Empty, Failure, Data) with typed AppFailure payloads.
4. Atomic Mutations: Team mutations (join, leave, vote, lock, pool-invite) run within Firestore Transactions to guarantee concurrency safety.

Directory Structure & Layer Boundaries:

Directory	Layer / Responsibility	Key Classes / Enforcements
lib/models/	Domain Data & State Model	Student, Team, Vote, AppNotification, ViewState<T>, AppFailure
lib/theme/	Design Tokens & System	ColorScheme, AppColors, TextTheme, AppSpacing, AppRadius
lib/widgets/	Reusable UI Components	AppButton, AppTextField, AppCard, AppDialog, LoadingState, ErrorState
lib/services/	Abstract Service Contracts	AuthService, TeamRepository, NotificationRepository, ConfigService, StorageService
lib/firebase/	Concrete Cloud Backends	FirebaseAuthService, FirestoreTeamRepository, FirebaseStorageService, FirebaseRemoteConfigService
lib/viewmodels/	Presentation Logic & State	SignInViewModel, BrowseViewModel, TeamDetailViewModel, LockReviewViewModel
lib/screens/	Pure Declarative Views	StatelessWidget / ConsumerWidget rendering ViewState<T>, zero SDK calls
lib/app/	App Shell & Routing	go_router with auth redirects, providers.dart composition root

4. DESIGN SYSTEM IN CODE & ACCESSIBILITY

Design Tokens, Reusable Widget Library & WCAG Validation

All visual tokens originate from Figma PRM_LAB2 page 02 Design Tokens and are codified in lib/theme/. No hardcoded hex colors, font sizes, or paddings exist in application screens.

COLOR & TYPOGRAPHY TOKENS

- Primary: #0284C7 (FPT Navy Accent)
- Success: #15803D (Verified / Locked)
- Warning: #B4530A (Deadlines / Warnings)
- Error: #B91C1C (Negative / Blocked)
- Surface: #FFFFFF / Background: #F8FAFC
- Type Scale: Roboto (12sp caption, 14sp body, 16sp title, 20sp headline, 26sp display)

SPACING, RADIUS & ELEVATION

- Spacing: AppSpacing.xs (4dp), sm (8dp), md (16dp), lg (24dp), xl (32dp)
- Corner Radius: sm (4dp), md (8dp), lg (12dp), pill (999dp)
- Elevation: AppElevation.none (0), card (1dp), dialog (4dp)
- Min Touch Target: 48dp x 48dp minimum for all buttons and interactive chips

Reusable Component Library (lib/widgets/):

Component	File Location	Variants & States Supported	Accessibility Feature
AppButton	app_button.dart	Primary, Secondary, Destructive, Text x Default, Pressed, Loading, Disabled	48dp target, Semantics button label
AppTextField	app_text_field.dart	SearchField, InputField x Default, Focused, Filled, Error	High contrast outline, clear icon button
AppCard	app_card.dart	Default, Outlined, SelectableCard (Radio check)	High contrast border, readable elevation
State Widgets	states.dart	LoadingState (Skeleton/Spinner), EmptyState (Icon+Action), ErrorState (Retry)	Descriptive feedback, no spinner forever
Status Indicators	status.dart	StatusBadge, InfoBanner, SlotRow, AppProgressBar	Icon + text (never color alone - WCAG 1.4.1)

5. CLOUD BACKEND DESIGN FIREBASE SERVICES (PART 1)

Authentication, Cloud Firestore & Cloud Storage Architecture

5.1. Firebase Authentication & Roster Authorization

 Implementation: FirebaseAuthService utilizes Google Sign-In and creates Firebase credentials via GoogleAuthProvider.credential().
 Session Stream: Listens to FirebaseAuth.instance.authStateChanges() and maps users to domain Student objects.
 Centralized Roster Allowlist: To accommodate real university scenarios where students (K19+) use individual email accounts, authentication is validated against Firestore allowedEmails/{email}. Any authenticated Google account not present with enabled: true is rejected with "This account is not on the class roster".
 Failure Handling: Graceful handling for cancelled sign-in, wrong domain, and offline network state.

5.2. Cloud Firestore & Concurrency Safeguards

 Implementation: FirestoreTeamRepository manages collections teams, waitingPool, and configuration.
 Transaction Safety: Roster joins, leaves, votes, and lock operations execute in runTransaction. If another user fills a team or casts a vote concurrently, the transaction aborts and retries safely.
 Waiting Pool Recovery: In Flow 3, inviteFromPool queries candidate documents, verifies assigned == false within the transaction, appends the member, and atomically marks assigned: true.
 Security Rules (firebase/firestore.rules): Strictly enforces isRosterMember() and limits updates to 8 whitelisted roster fields (GR-01 fix). No student can modify team metadata (supervisor, capacity, topic).

5.3. Firebase Cloud Storage & User Isolation

 Implementation: FirebaseStorageService generates team confirmation summary PDFs and uploads them to reports/{uid}/{fileName}.
 Progress & URL: Emits UploadProgress stream tracking byte transfer percentage (0% to 100%), then retrieves signed download URL.
 Security Rules (firebase/storage.rules): Grants read/write exclusively to request.auth != null && request.auth.uid == userId. Cross-user access is strictly forbidden by server rules.

6. CLOUD BACKEND DESIGN FIREBASE SERVICES (PART 2)

Cloud Messaging (FCM), Crashlytics, Remote Config & Analytics Plan

6.1. Firebase Cloud Messaging (FCM) Across 3 App States

 Permission: Automatically requests POST_NOTIFICATIONS consent on Android 13+ devices.
 Three Lifecycle States:

1. Foreground: onMessage captures notifications and persists them directly into the Alerts inbox.
2. Background: onBackgroundMessage runs isolated handler in main.dart writing directly to Firestore students/{uid}/notifications.
3. Terminated: getInitialMessage launches the app and onMessageOpenedApp navigates to /browse/team/{teamId}.

 Deep Linking: Tapping a notification with teamId payload navigates directly to the relevant Team Detail screen.

6.2. Firebase Crashlytics & Diagnostic Telemetry

 Fatal Errors: FlutterError.onError and PlatformDispatcher.instance.onError forward unhandled engine errors to Crashlytics.
 Handled Non-Fatal: ProfileViewModel.sendNonFatal simulates failed network lookups, with custom keys screen=profile and student_cohort (set at sign-in with the user ID; empty when the student profile has no cohort); team_id is set on team screens.
 Test Crash Action: Dedicated Profile button invokes FirebaseCrashlytics.instance.crash() for live verification.

6.3. Firebase Remote Config & Zero-Rebuild Realtime Updates

 In-App Defaults: ConfigKeys.defaults defines max_team_members=5, enable_waiting_pool=true, registration_closed=false.
 Real-Time Synchronization: onConfigUpdated automatically activates published values and emits through changes stream.
 Runtime change without reinstalling: publishing registration_closed=true in the Console switches Home to the closed banner and, while enable_waiting_pool is true, shows the Waiting Pool entry for students without a team.

6.4. Google Analytics Event Plan (Privacy-by-Design)

Tracks 10 events (login, logout, search_teams, view_team, create_team, join_team, cast_vote, invite_pool, lock_roster, export_report). Parameters carry IDs and counts only: search_teams sends query_length, never the search text. Every E2E run saves the events it caused from Logcat (analytics.txt); login with setUserId, search_teams and cast_vote are also shown in DebugView (evidence/debugview/).

7. AUTOMATED E2E TESTING WITH GOOGLE ARTEMIS

13 Test Cases, 3-Run Reliability Benchmark & Ground-Truth Assertions

Google ARTEMIS is an open-source AI agent that navigates the real Android app using multimodal models. The suite in e2e_test/ uses pytest-asyncio and asserts on the Semantics accessibility tree and Logcat rather than status=="SUCCESS".

Case ID	Flow Tested	Goal & Natural-Language Instruction	3-Run Pass	Avg ARTEMIS time
TC-01	Auth / Error	Sign in with unauthorized account -> Verify error banner	3/3 (100%)	62.9s
TC-02	Auth / Success	Sign in with roster account -> Verify Home greetings & deadline	3/3 (100%)	42.8s
TC-03	Flow 1 / Search	Browse tab -> Filter AI/ML & Open Slots -> Search "drone"	3/3 (100%)	45.4s
TC-04	Flow 1 / Empty	Search non-existent term -> Verify empty state -> Clear filters	3/3 (100%)	94.1s
TC-05	Flow 1 / Offline	Disable network via ADB -> Observe error -> Reconnect & retry	3/3 (100%)	21.8s
TC-06	Flow 1 / Join	Find FinTech Guard -> Join Team -> confirm -> My Team shows 3 of 5	3/3 (100%)	45.3s
TC-07	Flow 1 / Filled	Open team -> Concurrently fill team -> Verify filled banner	0/3 (0%)	-
TC-08	Flow 2 / Voting	Leader voting -> Verify tallies hidden before casting vote	0/3 (0%)	-
TC-09	Flow 2 / Elect	Vote for yourself -> Submit -> Verify elected banner & Firestore	0/3 (0%)	-
TC-10	Flow 3 / Lock	Lock blocked (<4) -> Invite from pool -> Recover & lock	0/3 (0%)	-
TC-11	Flow 3 / Native	Tap Share Team Link -> Verify Android Share Sheet intent	0/3 (0%)	-
TC-12	Alerts / Undo	Decline team invite -> Click Undo in snackbar -> Restore card	3/3 (100%)	42.2s
TC-13	Profile / Export	Export & Upload Team PDF -> Verify progress & Storage URL	3/3 (100%)	30.1s

Result: 24 of 39 runs passed. Failed runs and their causes (including Gemini free-tier quota exhaustion) are listed per trace ID in docs/test-plan.md section 3.

Exploratory Testing Session (ARTEMIS Pro profile):
Not run yet. See docs/test-plan.md section 4.

8. AI-ASSISTED CODE REVIEW & AUDITABLE DECISIONS

Pull Request #8 Code Review via Google Gemini (gemini-3.5-flash-lite)

The team configured an automated GitHub Actions workflow (.github/workflows/gemini-review.yml) that submits pull request diffs to Google Gemini. 5 findings across 3 categories were recorded with rigorous technical decisions.

ID	Category · Severity	Location	Gemini Finding Summary	Decision	Commit
GR-01	Security · High	firestore.rules:57	isSelfJoin only checks roster size, allowing malicious modification of team metadata (supervisor, capacity).	Accepted (Widened to all 5 team mutations)	c9543a3
GR-02	Concurrency · Med	firestore_team_repo:269	Querying waiting pool outside transaction could cause 2 teams to target the same student.	Rejected (In-transaction re-read & atomic assignment prevent race)	X
GR-03	Reliability · Med	backend.py:42	Missing Firebase CLI login raises bare KeyError instead of actionable guidance.	Accepted (Provides explicit firebase login instruction)	ee6ec78
GR-04	Reliability · Low	remote_config_svc:24	Catch block suppresses Remote Config startup fetch error message.	Accepted (Adjusted to log error and continue with in-app defaults)	d6926d1
GR-05	Reliability · Med	gemini_review.py:61	Slicing error body to 2000 chars might split UTF-8 multi-byte characters.	Rejected (False positive: decode occurs before string slicing)	X

Detailed Analysis of Key Review Findings:

X GR-01 (Crucial Security Fix): Demonstrated that AI review excels at spotting authorization boundary gaps. The fix strictly enforces after.diff(before).affectedKeys().hasOnly([...]) across all 5 team mutation types.
X GR-02 (Critical Human Judgment): Rejecting GR-02 proved that engineers must not blindly trust AI suggestions. Firestore Client SDK does not support queries inside transactions; re-reading the candidate document with tx.get() inside the transaction is the canonical architectural solution.

9. VERIFICATION EVIDENCE MATRIX & CONSOLE PROOF

Comprehensive Artifacts Linking Requirements to Verifiable Evidence

Each row is marked VERIFIED only when its screenshots or files exist in the repository at the time this PDF is generated.

Requirement	Console / Device Evidence	Repository Location	Verification Status
Google Sign-In	Authentication Users tab with the Google-provider accounts and UIDs	evidence/console/console_auth_users.jpg	VERIFIED
Firestore Roster & Rules	allowedEmails roster, deployed team and waitingPool rules	evidence/console/	VERIFIED
Storage Isolation	reports/{uid}/ PDFs uploaded by the app and the deployed storage.rules	evidence/console/	VERIFIED
FCM 3 States	Test message received in foreground, background and terminated states	evidence/fcm/	VERIFIED
Analytics DebugView	login with user ID, search_teams and cast_vote with their parameters	evidence/debugview/	VERIFIED
Crashlytics Reports	Crash dashboard and a non-fatal issue with keys screen and student_cohort	evidence/crashlytics/	VERIFIED
Remote Config	App before, value published in Console, app after without reinstalling	evidence/remote_config/	VERIFIED
ARTEMIS E2E	JUnit XML, per-run result.json, final screenshot, screen text, Logcat events (24/39 runs passed)	e2e_test/evidence/	VERIFIED
AI Code Review	Pull request #8 review comments, docs/code-review.md, 3 fix commits	docs/code-review.md	VERIFIED
Design Traceability	Figma frames next to app screenshots, deviations with reasons	docs/design-traceability.md	VERIFIED

10. CHALLENGES ENCOUNTERED & LESSONS LEARNED

1. Cross-Service Storage Rules: Initially attempted to validate user team membership inside storage.rules using firestore.get(). This caused HTTP 403 errors because Firebase Storage does not support cross-service Firestore lookups. Fixed by scoping rules directly to authenticated UID: reports/{userId}/{allPaths=**}.
2. AI Review Quota Throttling: Multi-turn agentic review tools exhausted Gemini API quotas. Solved by engineering a single-request script (.github/scripts/gemini_review.py) that transmits the entire git diff in one structured prompt.
3. Asynchronous Provider Lifecycle: Fixed a subtle bug where notifyListeners() was called on SignInViewModel after being unmounted during async Google sign-in transitions.

10. DISCLOSURE OF AI TOOLS & SUBMISSION CHECKLIST

Academic Integrity Statement & Final Verification

10.1. Disclosure of AI Tools Used (Handout Section 10 Compliance)

AI Tool / Service	Model / Version	Purpose in Project	Human Review & Verification
Google Gemini (CLI/Actions)	gemini-3.5-flash-lite	Pull Request #8 automated code review	All 5 findings evaluated manually; 3 accepted, 2 rejected
Google ARTEMIS	Commit 351ca84, Flash & Pro Profile	E2E automated test execution & exploratory testing	Device assertions verified via Semantics tree & Logcat
Antigravity / Claude	Claude Opus 5.5 / Gemini 3.8 Flash	Pair programming, MVVM architecture & report drafting	Code reviewed line-by-line; tested on Android API 36 emulator

10.2. Final Submission Checklist & Verification

- [ ] Submission repository name PRM393_Lab03_<StudentID>: the GitHub repository is still DEVORGANIZE222/Lab3_Cloud-Enabled_Mobile_App.
- [x] MVVM layering enforced: Zero Firebase SDK imports in lib/screens/ or lib/widgets/.
- [x] All 6 Firebase cloud services operational with explicit failure and offline handling.
- [x] Storage rules enforce strict per-user UID isolation (no public allow read, write: if true).
- [x] Analytics event plan implemented with 10 events, shown in DebugView, no PII in parameters.
- [x] FCM test message received in foreground, background and terminated states.
- [x] Crashlytics reports both deliberate test crash and non-fatal error with custom keys.
- [x] Remote Config updates the UI at runtime without reinstalling the APK.
- [x] ARTEMIS E2E suite covers 13 test cases x 3 runs: 24/39 runs passed.
- [ ] ARTEMIS Pro exploratory session recorded in docs/test-plan.md section 4.
- [x] AI Code Review on PR #8 documented with 5 findings and 3 traceable fix commits.
- [x] Final Report PDF (10 pages) and Presentation Slides PDF (15 slides) generated.

END OF REPORT ☒ DEVORGANIZE222 ☒ PRM393 LAB 3